

Set 6

Below you can find the instructions for each exercise in **Set 6**.

You can create a workspace for your **Set6** folder in **VS Code** by selecting **Open Folder...** and choosing your **Set6** folder. This allows you to access and manage all your exercise files in one place.

Open the **.c** file you want to run using **VS Code** or any other text editor you prefer (for example, Notepad on Windows). You can edit your C file directly in any of these editors.

When you finish writing your code, you need to **compile it**. To do this, open **w64devkit** and navigate to the **exercise folder** you are working on within your **Set6** directory. For detailed steps on how to navigate to the folder and compile your program, refer to the *"Getting started with LEU483"* guide on Canvas.

In this exercise no new C-concepts are introduced, but the more general concept of *pseudocode* is introduced and used in some exercises.

Pseudocode

In this exercise set, we use pseudocode to describe what you are supposed to do. Pseudocode is a plain language description of the steps of a code. It usually resembles actual code, but it is not code, which can be run on a machine. Since language and/or environment dependent features are omitted, it is usually easier to read. Text books and scientific publications rely on pseudocode to document algorithms. For a further overview of pseudocode, see the Wikipedia article: <https://en.wikipedia.org/wiki/Pseudocode>.

Pseudocode for 3 elementary sort algorithms: <https://faculty.cs.niu.edu/~hutchins/csci241/sorting.htm>

1 Count unique

Open: `ex_count_unique/main.c`

Many algorithms applied to arrays require the array to be sorted first. In this exercise you will sort an array, then count the number of unique elements in the sorted array.

Task:

- Implement the function `selection_sort` that sorts an array with n elements (size of array is n) in ascending order. See pseudocode below

```
for i=0 to i=n-1 do:
    k = i
    for j=i+1 to j=n-1 do:
        if array(j) < array(k) do:
            k = j
    temp = array(i)
    array(i) = array(k)
    array(k) = temp
```

- Sort the array by calling `selection_sort` in `main`

- Implement the function `count_unique`. Try to write pseudocode on paper for this task before implementing it
- Count the number of unique elements in the array by calling `count_unique` in `main`

Program output:

```
Unsorted array: 14, 3, 1, 9, 12, 5, 1, 15, 11, 10, 1, 4, 7, 9, 14, 8, 5
Sorted array:   1, 1, 1, 3, 4, 5, 5, 7, 8, 9, 9, 10, 11, 12, 14, 14, 15
There are 12 unique numbers
```

2 Binary search

Open: `ex_binary_search/main.c`

In this exercise you will sort an array using the insertion sort algorithm and then find the index of a specific value using the binary search algorithm.

Task:

- Implement the function `insertion_sort` that sorts an array of size `n` in ascending order. See pseudocode below

```
for i=1 to i=n-1 do:
    j=i
    while (j > 0) and (array(j) < array(j-1)) do:
        temp = array(j)
        array(j) = array(j-1)
        array(j-1) = temp
        j = j-1
```

- Sort the array by calling `insertion_sort` in `main`
- Implement the function `binary_search` that finds the index of some value in an array of size `n`. If the value doesn't exist in the array the function should return -1, see pseudocode below

```
i = 0          (first index of array)
j = n - 1      (last index of array)
while i <= j do:
    k = (i + j)/2
    if value to seek < array(k) do:
        j = k - 1
    else if value to seek > array(k) do:
        i = k + 1
    else do:
        break
if value to seek equals array(k) do:
    return k
```

```
else do:
    return -1
```

- Find the index of the value 11 (stored by the variable `value`) by calling `binary_search` in `main`
- Try to search for a value that does not exist in the array e.g. by changing `value` to 6. The function should return index -1

Program output:

```
(run searching for value 11)
Unsorted array: 14, 3, 2, 9, 12, 5, 1, 15, 11, 16, 4, 7, 8, 17
Sorted array:   1, 2, 3, 4, 5, 7, 8, 9, 11, 12, 14, 15, 16, 17
Value 11 is at index 8

(run searching for value 6)
Unsorted array: 14, 3, 2, 9, 12, 5, 1, 15, 11, 16, 4, 7, 8, 17
Sorted array:   1, 2, 3, 4, 5, 7, 8, 9, 11, 12, 14, 15, 16, 17
Value 6 is at index -1
```

3 π series

Open: `ex_pi/main.c`

Make a function that calculates π using the Madhava-Leibniz's formula, see

https://en.wikipedia.org/wiki/Leibniz_formula_for_%CF%80. This formula expresses $\pi/4$ as an alternating series according to

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots$$

The function should add terms to the series until the absolute value of the next term is smaller than a limit value given as argument to the function (this next term shall still be added).

Task:

- Implement the function `calculate_pi` which approximates π by adding all terms in the Madhava-Leibniz series until the limit given above
- Run the code using a limit of 0.01
- Change the limit to be 0.001 and run again

Program output:

```
PI approximated to 3.161199      (run with limit = 0.01)
PI approximated to 3.143589      (run with limit = 0.001)
```

4 Fibonacci

Open: ex_fibonacci/main.c

The Fibonacci sequence consists of numbers, which are the sum of the two previous elements. Here, the first and the second elements are defined as 0 and 1 respectively. Mathematically speaking the n -th Fibonacci number $F(n)$ is calculated by $F(n)=F(n-2)+F(n-1)$. So the Fibonacci sequence looks like this: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Task:

- Implement the following pseudocode, which prints the Fibonacci sequence:

Pseudocode:

```
a=0
b=1
print(a,b)
for(i=1 to 15):
    t=a+b
    a=b
    b=t
    print(t)
```

5 Monte Carlo

Open: ex_montecarlo/main.c

In this exercise, you will approximate Pi using a so called Monte Carlo method. Monte Carlo methods use random numbers to get numerical results. The ratio between the area of a circle with radius r and a square with an edge length of $2r$ is $\pi/4$. This ratio is unchanged if we consider only a quarter or half of the square and the circle. For simplicity we will assume a radius $r=1$ and only consider the first quadrant of a coordinate system (i.e., $x \geq 0, y \geq 0$).

Pseudocode:

```
inCircleArea = 0
inSquareArea = 0
for(i=1 to 100):
    x = random value from [0,1]
    y = random value from [0,1]
    if (x,y) within the circle:
        inCircleArea = inCircleArea + 1
    inSquareArea = inSquareArea + 1
pi = 4.0*inCircleArea/inSquareArea
print pi
```

Task:

- Describe in your own words what this algorithm does (max. 4 short sentences).
- Implement the pseudocode in C and compare your result with the actual value of Pi.
- Increase the number of randomly chosen points to 1000, 10000, and 100000. How does the value you calculated change compared to the actual value of Pi?

Hints:

- For comparison to the actual value of Pi, either use the definition in `math.h` or `pi=4*atan(1)`.

6 Text analysis

Open: `ex_text_analysis/main.c`

In this exercise, you are supposed to write a program that counts words and characters in a text file. A word is any collection of letters, including joint words (e.g. Close-up), but not numbers. Characters include all letters, numbers, special characters (e.g. - ,), and spaces.

You should use separate functions to open the file, read its content, count words, and count characters. Some of these you might have written for earlier exercises, and we encourage you to re-use them. Develop and test your program on the file `test.txt` included in the `ex_text_analysis` directory.

Example output:

For the second example below, you have to add the line "Stop-line" to `text.txt`.

```
File content:                (first example)
```

```
-----
```

```
Line 1
```

```
This is the second line
```

```
12324 cars
```

```
A last line
```

```
-----
```

```
Characters in file: 50
```

```
Words in file: 10
```

```
File contents:              (second example)
```

```
-----
```

```
Line 1
```

```
This is the second line
```

```
12324 cars
```

```
A last line
```

```
Stop-line
```

```
-----
```

```
Characters in file: 59
```

```
Words in file: 11
```

Hints:

- You have used `fgets` to read user input, but it can also read files.

- `fgets` returns `NULL` when it encounters the end of a file.
- Remember that you have to specify the full path to the text file.

Obs! Om filen `test.txt` inte hittas när du kör koden, prova att ange hela sökvägen till filen. E.g.
`/home/yourname/../../set6/ex_text_analysis/test.txt`

Bonus:

When the above program is completed, you can try to modify it so that it is possible for the user to supply the path to the file as a command line argument at runtime. This is **outside the scope of the course**, but still a fun exercise for the interested.

7 People

Open: `ex_people/main.c`

In this exercise, you will read a file containing the names and ages of several people. You will then sort and extract information on the people.

Task:

- Create a struct called 'person' that stores the name of a person and their age.
- Create an array called 'people' that holds person structs. Read the information from the file `people.txt` into this array.
- Sort the array 'people' by name, i.e. alphabetically.
- Search the array 'people' for the person named 'Elsa' and print her age.
- Resort the array 'people' by age, lowest to highest.
- Print the names of all the people over the age of 30.

Obs! Om filen `people.txt` inte hittas när du kör koden, prova att ange hela sökvägen till filen. E.g.
`/home/yourname/../../set6/ex_people/people.txt`

Example output:

```
Age of Elsa is 6
People older than 30:
Erik
Gustav
Karl
Oskar
```